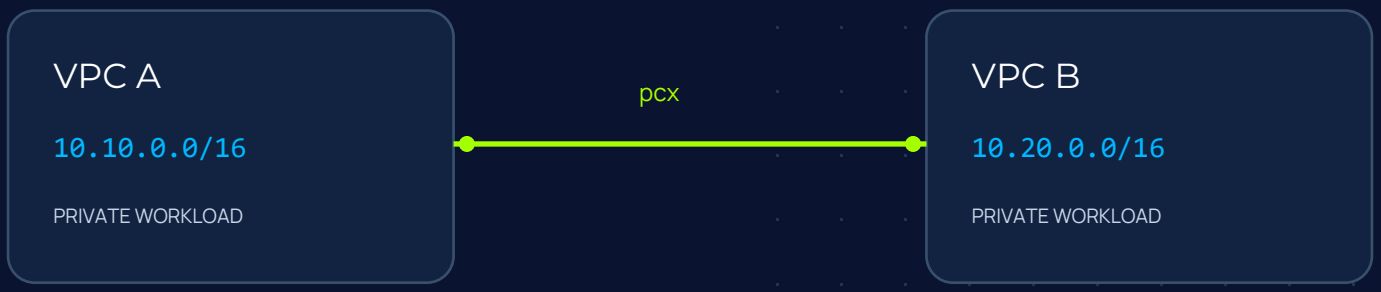
BUILD / TEST / EXPLAIN

AWS

VPC Peering

Connect two VPCs.
Prove private connectivity.

A guided networking project with EC2, custom route tables, security groups and a bastion host.



2 VPCs / 3 EC2 INSTANCES / 1 PRIVATE CONNECTION

AWS ARCHITECT ASSOCIATE

Name: _____ Batch: _____

01 / PROJECT BRIEF

Your mission

Build a working private network connection and explain why each route and security rule is needed.

EWB has an application in VPC A and a second workload in VPC B. Both application servers must stay private. Create VPC peering so they can exchange traffic using their private IPv4 addresses. Use one public bastion in VPC A for administration.

What you will demonstrate

- Plan non-overlapping VPC and subnet CIDRs.
- Create public and private subnets with explicit route table associations.
- Activate peering, configure return routes and allow only the required inbound traffic.
- Show a failed test before peering, successful tests after peering and a controlled failure.

Before you begin

Use an AWS lab account with permission to manage VPC, EC2 and key pairs. Know CIDR basics, routes and SSH. Keep all resources in one account and one Region (for example, Mumbai / ap-south-1). Use Amazon Linux 2023 and a local OpenSSH client; Windows students can use WSL.

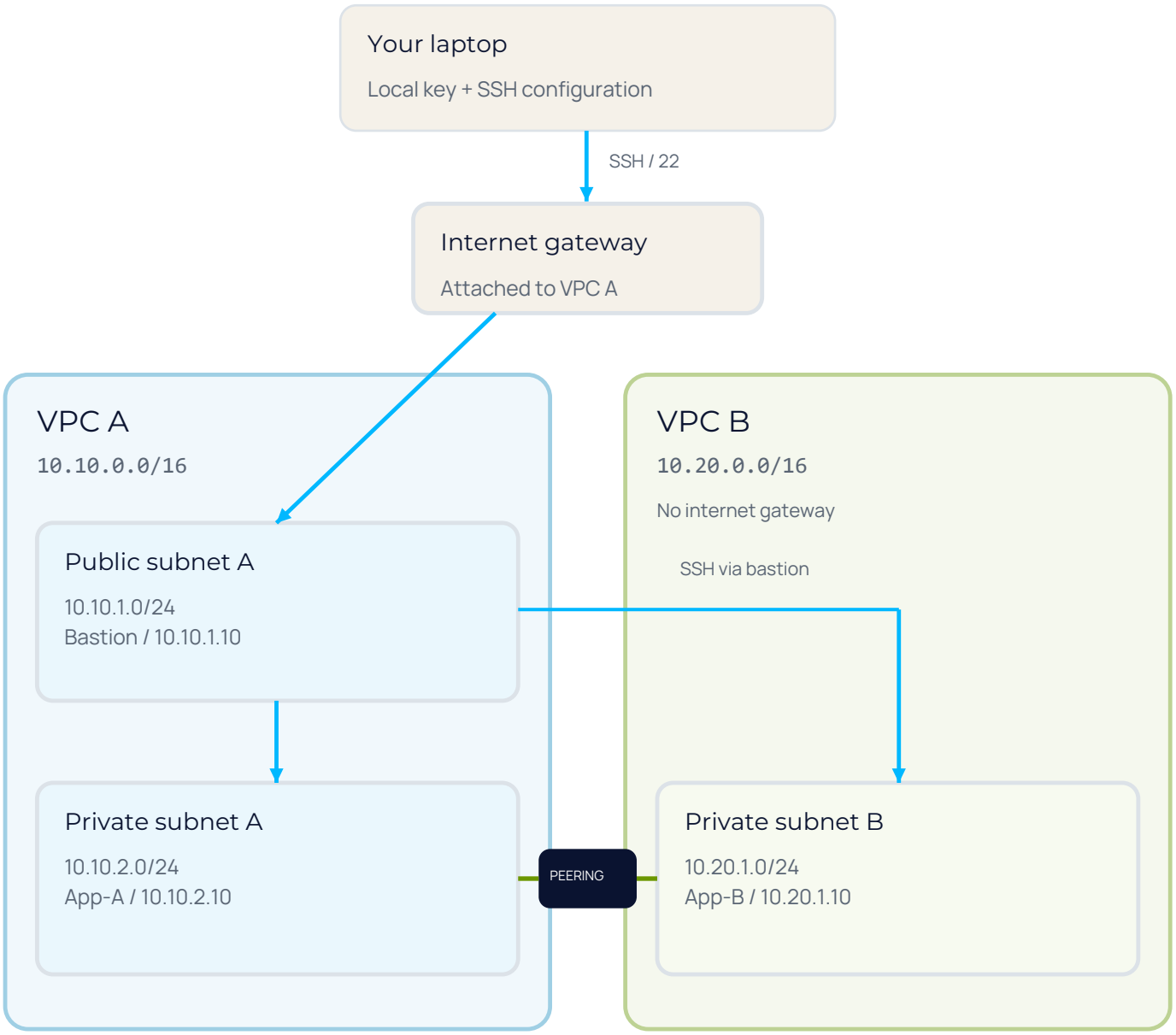
> Plan costs before launch
> EC2, EBS and public IPv4 usage can incur charges. Confirm your account credits and current pricing. This lab needs no NAT gateway. Keep resources only for the lab and complete the cleanup on page 14.

Find it here	Pages
Architecture and IP plan	3-4
Network, security and EC2 setup	5-7
SSH access, peering and routes	8-10
Validation and troubleshooting	11-12
Submission, evaluation and cleanup	13-14

02 / ARCHITECTURE

The network you will build

One internet entry point. Two private workloads. A direct peering connection between their VPCs.



Administration: laptop > bastion > App-A or App-B using SSH ProxyJump.
Project traffic: App-A < > App-B over private IPs and the peering connection.

Diagram is conceptual. The bastion opens a separate SSH connection to the destination. Peering does not provide VPC B with internet access through VPC A.

03 / RESOURCE PLAN

Use this address plan

Keep the names consistent so you can match screenshots, routes and terminal output.

Resource / Name tag	VPC or subnet	IPv4 CIDR / address
ewb-vpc-a	VPC A	10.10.0.0/16
ewb-a-public	VPC A / public subnet	10.10.1.0/24
ewb-a-private	VPC A / private subnet	10.10.2.0/24
ewb-vpc-b	VPC B	10.20.0.0/16
ewb-b-private	VPC B / private subnet	10.20.1.0/24
ewb-bastion	ewb-a-public	10.10.1.10 + public IPv4
ewb-app-a	ewb-a-private	10.10.2.10 / no public IP
ewb-app-b	ewb-b-private	10.20.1.10 / no public IP

Assign the listed private IPv4 addresses during instance launch. If you choose AWS-assigned addresses instead, replace every instance IP and /32 security rule in this workbook with your actual values.

Name the remaining resources

Internet gateway: ewb-igw-a
Route tables: ewb-rt-a-public, ewb-rt-a-private, ewb-rt-b-private
Security groups: ewb-sg-bastion, ewb-sg-app-a, ewb-sg-app-b
Peering connection: ewb-a-b-peer
EC2 key pair: ewb-lab

> Tag your work
> Add Project = EWB-VPC-Peering and Student = your name to the resources you create. Record your actual VPC, subnet, route table, instance and peering IDs.

Why these CIDRs? 10.10.0.0/16 and 10.20.0.0/16 do not overlap. Each /24 subnet belongs to its own VPC range. Different subnet ranges alone do not fix overlapping VPC CIDRs.

04 / BUILD THE NETWORK

Create VPC A

Use VPC only in the creation wizard so you control every subnet and route.

1. Create the first VPC
   In the VPC console, choose Your VPCs > Create VPC > VPC only. Name it ewb-vpc-a, enter 10.10.0.0/16, use no IPv6 CIDR and leave tenancy as Default.

2. Create two subnets
   Choose Subnets > Create subnet. Select VPC A. Create ewb-a-public (10.10.1.0/24) and ewb-a-private (10.10.2.0/24). Use the same Availability Zone for this lab where available.

3. Attach the internet gateway
   Create ewb-igw-a in Internet gateways. Select it, choose Actions > Attach to a VPC, and select ewb-vpc-a. An internet gateway belongs to the VPC; it is a route target in the route table.

4. Create the public route table
   Create ewb-rt-a-public in VPC A. Under Subnet associations, explicitly associate ewb-a-public. Under Routes > Edit routes, add 0.0.0.0/0 with target ewb-igw-a. Keep the existing local route.

5. Create the private route table
   Create ewb-rt-a-private in VPC A and explicitly associate ewb-a-private. Keep its local route. Do not add an internet default route. Leave subnet auto-assign public IPv4 disabled for the private subnet.

> Checkpoint 01
> Capture the two subnet associations and the public default route. The private route table should currently contain only its local VPC route.

Enable a public IPv4 address only for the bastion when you launch it on page 7. A public subnet needs an internet-gateway route, and the instance needs a public address for direct internet access.

05 / NETWORK AND SECURITY

Build VPC B. Set the rules.

Keep both application servers private and scope inbound access to the lab hosts.

Create ewb-vpc-b with CIDR 10.20.0.0/16. Create ewb-b-private with CIDR 10.20.1.0/24. Create ewb-rt-b-private, explicitly associate this subnet and retain its local route. Keep public IPv4 auto-assignment disabled. VPC B needs no internet gateway.

Create these inbound security group rules

Security group	Type / port	Source
ewb-sg-bastion VPC A	SSH / TCP 22	Your current public IPv4 /32
ewb-sg-app-a VPC A	SSH / TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-a VPC A	Custom ICMP - IPv4 Echo request	10.20.1.10/32 (App-B)
ewb-sg-app-b VPC B	SSH / TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-b VPC B	Custom ICMP - IPv4 Echo request	10.10.2.10/32 (App-A)

For Echo request, use ICMP type 8, code 0 if the console asks. Keep default outbound allow-all rules for this beginner lab. Security groups are stateful, so replies to permitted traffic are allowed automatically. Use the default allow-all network ACLs; custom ACLs need return traffic rules too.

> Port and source are different fields
> SSH uses port 22. The private IP or CIDR goes in Source, never in Port. App-B receives the bastion's private source IP when you access it through ProxyJump.

The /32 rules intentionally permit only individual lab hosts. If an IP changes, update its rules. Do not expose SSH or ICMP to 0.0.0.0/0.

06 / COMPUTE

Launch the three instances

Use the same downloaded key pair and Amazon Linux username throughout the lab.

1 Create and store your key pair
In EC2, create an RSA key pair named ewb-lab and download the .pem file. Store it on your own laptop. Use it for all three instances. Never upload or paste the private key onto an EC2 instance.

2 Use a consistent instance configuration
Select Amazon Linux 2023 and an instructor-approved small x86 instance type, such as t3.micro. Check the displayed price and your account credits. Keep the root disk small and enable delete-on-termination for that lab volume.

Instance	Network settings	Security group
ewb-bastion	VPC A / ewb-a-public Private IP: 10.10.1.10 Auto-assign public IP: Enable	ewb-sg-bastion
ewb-app-a	VPC A / ewb-a-private Private IP: 10.10.2.10 Auto-assign public IP: Disable	ewb-sg-app-a
ewb-app-b	VPC B / ewb-b-private Private IP: 10.20.1.10 Auto-assign public IP: Disable	ewb-sg-app-b

Under Network settings > Edit, select the VPC and subnet. In the advanced network interface settings, set the primary private IPv4 to the address above. Choose the existing security group for that VPC. Launch each instance and wait for it to run and pass its status checks.

> Checkpoint 02
> Record all three instance IDs and private IPs, plus the bastion public IPv4. Confirm the Public IPv4 address field is empty for App-A and App-B.

No package download is required for the base lab. Private instances have no internet egress. A failed package update there does not mean peering is broken.

07 / ADMINISTRATION

Connect without copying keys

Run all SSH commands on your laptop. ProxyJump uses the bastion as the connection path.

Move the key to ~/.ssh/ewb-lab.pem in WSL, macOS or Linux. Run chmod 400 ~/.ssh/ewb-lab.pem. Create or edit ~/.ssh/config with the entries below. Replace BASTION_PUBLIC_IP with the bastion's current public IPv4. Keep the private IPs only if they match your instances.

LOCAL FILE / ~/.SSH/CONFIG

```
Host ewb-bastion
    HostName BASTION_PUBLIC_IP
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes

Host ewb-app-a
    HostName 10.10.2.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion

Host ewb-app-b
    HostName 10.20.1.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion
```

Test ssh ewb-bastion first. Use exit to return to your laptop, then run ssh ewb-app-a. Both should connect before peering, using VPC A's local route. Verify the destination host before accepting a first-time host key prompt.

> Capture the baseline now
> Inside App-A, run ping -c 4 10.20.1.10. Before peering and peer routes, it should receive no replies. Save this result. App-B SSH access is expected to work only after pages 9-10 are complete.

08 / ESTABLISH PEERING

Request. Accept. Activate.

The peering connection is the link. Route tables and security groups determine whether traffic can use it.

1 Create the peering request
In the VPC console, choose Peering connections > Create peering connection. Use the settings below. This project uses the same AWS account and Region on both sides.

Field	Value
Name	ewb-a-b-peer
VPC ID (Requester)	Your ewb-vpc-a ID
Account	My account
Region	This Region
VPC ID (Accepter)	Your ewb-vpc-b ID

2 Accept the request
Select the new connection. Choose Actions > Accept request and confirm. You must accept even when both VPCs belong to your own account.

3 Record the active connection
Wait until Status shows Active. Record the actual peering ID (pcx-...). Confirm requester VPC A, accepter VPC B and their CIDRs before continuing.

> Checkpoint 03
> Save a screenshot showing the connection name, Active status and both VPC IDs. Peering alone will not make ping or SSH succeed; add the routes on the next page.

VPC peering can also span AWS accounts or Regions. Those are extension exercises. This workbook uses private IPs directly and does not require changing the peering DNS resolution options.

09 / ROUTING

Add routes on both sides

Open each named route table, confirm its subnet association, then choose Routes > Edit routes.

In the tables below, pcx-ID means the actual ewb-a-b-peer connection. igw-ID means ewb-igw-a. Keep the AWS-created local routes. Add only the missing routes.

ewb-rt-a-private / associated to ewb-a-private

Destination	Target	Purpose
10.10.0.0/16	local	Existing VPC A route
10.20.0.0/16	pcx-ID	App-A to VPC B

ewb-rt-b-private / associated to ewb-b-private

Destination	Target	Purpose
10.20.0.0/16	local	Existing VPC B route
10.10.0.0/16	pcx-ID	Return path to VPC A

ewb-rt-a-public / associated to ewb-a-public

Destination	Target	Purpose
10.10.0.0/16	local	Bastion to App-A
0.0.0.0/0	igw-ID	Bastion internet access
10.20.0.0/16	pcx-ID	Bastion SSH to App-B

> Why does the public subnet need a peering route?
> ProxyJump makes the bastion open a connection to App-B. Its subnet must know how to reach VPC B. The more specific 10.20.0.0/16 route is selected instead of the internet default route.

Checkpoint 04: save the routes and subnet associations for all three tables. A one-way route is insufficient: replies also need the route back.

10 / VERIFICATION

Prove the project works

Record real terminal output. The expected results below are acceptance criteria, not completed test results.

Test	Where / command	Expected result
01 / Baseline	App-A, before peering: ping -c 4 10.20.1.10	No replies
02 / Forward path	App-A, after routes: ping -c 4 10.20.1.10	4 replies; 0% packet loss
03 / Reverse path	App-B, after routes: ping -c 4 10.10.2.10	4 replies; 0% packet loss
04 / Private SSH	Laptop: ssh ewb-app-b	Shell on App-B through bastion
05 / Isolation	EC2 instance details: App-A and App-B	No public IPv4 on either host

Run a controlled failure

Temporarily remove only the App-B inbound ICMP Echo request rule from 10.10.2.10/32. Repeat the forward ping from App-A: replies should stop. Restore the exact rule and rerun the test: replies should resume. Keep the SSH rule in place.

IDENTIFY THE INSTANCE / RUN INSIDE EACH PRIVATE HOST

```
hostname
hostname -I
ping -c 4 PEER_PRIVATE_IP
```

Replace PEER_PRIVATE_IP with the other private instance address. Capture the command and the ping summary. If the first run loses a packet while the host becomes ready, wait briefly and repeat the four-packet test.

> Evidence of success
> Submit baseline failure, two-way ping success, App-B SSH through the bastion and the rule-removal failure followed by recovery. A screenshot of Active peering by itself does not prove reachability.

11 / TROUBLESHOOTING

Find the missing link

Check the instance state, the destination IP, the subnet route table and the security rule in that order.

Symptom	What to inspect
Bastion SSH times out	Its current public IP; your current public IP /32 in the SSH rule; public subnet association; 0.0.0.0/0 route to the attached IGW.
App-A SSH fails	App-A private IP; TCP 22 allowed from 10.10.1.10/32; correct key, ec2-user and running instance.
App-B SSH fails	Bastion public subnet needs 10.20.0.0/16 to pcx-ID; VPC B needs the return route; App-B SSH source is the bastion private IP.
Ping fails after acceptance	Both private subnet route tables; correct pcx-ID and Active state; Echo request rule on the destination; actual host IPs.
Peering creation fails	Check every CIDR associated with both VPCs for overlap, including secondary CIDRs.
Permission denied (publickey)	Check the correct key pair, local IdentityFile and ec2-user. Routing is working if you reached SSH authentication.
Failure with custom ACLs	Check both directions, ICMP rules and SSH return ephemeral ports. This lab assumes default allow-all ACLs.

Be ready to explain

1. Why is the return route needed?
2. Why is the App-B SSH source the bastion private IP?
3. Why does Active peering not guarantee a successful ping?
4. Can VPC B use VPC A's internet gateway over peering?

> Core concepts
> Peering is one-to-one and non-transitive. A-B plus A-C does not create B-C connectivity. Accepted connections can be deleted. Routes and security rules remain configurable; changing the paired VPCs requires a new connection.

12 / STUDENT DELIVERABLES

Submit a project you can defend

Your submission should let another student understand the design and reproduce the result.

Submission checklist

- [ ] Cover: your name, batch, project title, Region and completion date.
- [ ] Architecture diagram and inventory with actual resource IDs and private IPs.
- [ ] Subnet CIDRs and all three route tables, including their subnet associations.
- [ ] Active peering connection and inbound rules for each security group.
- [ ] Baseline failure, forward and reverse ping success, and App-B SSH proof.
- [ ] Controlled ICMP failure and recovery, with a short explanation.
- [ ] Cleanup evidence plus a short note on what you learned.

Submit one report PDF named YourName_EWB_VPC_Peering.pdf. Label every screenshot with the host, test and result. Include a short explanation of the forward path and return path. Leave private keys, passwords and access tokens out of your report.

Evaluation / 100 marks

Assessment area	Marks
Architecture, CIDRs and resource inventory	15
Subnets, internet gateway and correct route associations	20
Peering plus forward and return routes	20
Security rules and private SSH access	20
Positive and negative test evidence with explanations	15
Report clarity, viva and cleanup evidence	10

Before you submit: confirm both private hosts have no public IP, every test uses private destination IPs and the report shows your own execution results.

13 / FINISH RESPONSIBLY

Clean up. Keep the learning.

Save your evidence first. Remove only the resources you created for this project.

1 Remove compute and credentials
Terminate ewb-bastion, ewb-app-a and ewb-app-b. Confirm their lab EBS volumes are deleted; delete any leftover lab volumes or snapshots. Release any Elastic IP you created. Remove the unused lab key pair and local lab key if no longer needed.

2 Remove peering and network dependencies
Delete the peer routes and ewb-a-b-peer. Delete lab security groups after instance network interfaces disappear. Remove subnet associations as required, then delete the lab subnets and custom route tables. Detach and delete ewb-igw-a. Delete both lab VPCs.

3 Verify the account is clear
Filter by the project tag and check EC2, volumes, public IPv4 allocations and VPC resources in your lab Region. Check Billing for remaining chargeable resources; usage reporting can lag. Capture cleanup evidence.

Official AWS references

VPC peering concepts and limitations

Create a VPC peering connection

Update route tables for peering

Security groups for peered VPCs

EC2 key pair security

Amazon VPC pricing

Technical references checked on 16 October 2026. Console wording may vary. Keep the architecture, CIDRs, route targets and source rules consistent.

LEARN. ELEVATE. EMPOWER.